Chapter 4
Summation

I do hate sums.
There is no greater mistake than to call arithmetic an exact *science.*
There are . . . hidden laws of Number
which it requires a mind like mine to perceive.
For instance, if you add a sum from the bottom up,
and then again from the top down,
the result is always different.

-MRS. LA TOUCHE[10]

Joseph Fourier introduced this delimited z-notation in 1820,
and it soon took the mathematical world by storm.

-RONALD L. GRAHAM, DONALD E. KNUTH, and OREN PATASHNIK, *Concrete Mathematics (1989)*

One of the major difficulties in a practical [error] analysis
is that of description.
An ounce of analysis follows a pound of preparation.

-BERESFORD N. PARLETT, *Matrix Eigenvalue Problems* (1965)

[10]Quoted in Mathematical Gazette [730, 1924].

Sums of floating point numbers are ubiquitous in scientific computing. They occur when evaluating inner products, means, variances, norms, and all kinds of nonlinear functions. Although at first sight summation might appear to offer little scope for algorithmic ingenuity, the usual "recursive summation" (with various orderings) is just one of a variety of possible techniques. We describe several summation methods and their error analyses in this chap ter. No one method is uniformly more accurate than the others, but some guidelines can be given on the choice of method in particular cases.

4.1. Summation Methods

In most circumstances in scientific computing we would naturally translate a sum $\sum_{i=1}^{n} x_i$ into code of the form

```
s = 0
for i = 1:n
    s = s + x_i
end
```

This is known *as recursive summation.* Since the individual rounding errors depend on the operands being summed, the accuracy of the computed sum $\hat{s}$ varies with the ordering of the x_i. (Hence Mrs. La Touche, quoted at the beginning of the chapter, was correct if we interpret her remarks as applying to floating point arithmetic.) Two interesting choices of ordering are the increasing order $|x_1| \le |x_2| \le \ldots \le |x_n|$, and the decreasing order $|x_1| \ge |x_2| \ge \ldots \ge |x_n|$.

Another method is *pairwise summation* (also known as cascade summation, or fan-in summation), in which the x_i are summed in pairs according to

$$y_i = x_{2i-1} + x_{2i}, \quad i = 1 : \left\lfloor \tfrac{n}{2} \right\rfloor \quad (y_{(n+1)/2} = x_n \text{ if } n \text{ is odd}),$$

and this pairwise summation process is repeated recursively on the y_i, $i = 1:\lceil (n+1)/2 \rceil$. The sum is obtained in $\lceil \log_2 n \rceil$ stages. For $n = 6$, for example, pairwise summation forms

$$S_6 = ((x_1 + x_2) + (x_3 + x_4)) + (x_5 + x_6).$$

Pairwise summation is attractive for parallel computing, because each of the $\lceil \log_2 n \rceil$ stages can be done in parallel [573, 1988, §5.2.2].

A third summation method is the *insertion* met hod. First, the x_i are sorted by order of increasing magnitude (alternatively, some other ordering could be used). Then $x_1 + x_2$ is formed, and the sum is inserted into the list $x_2, \ldots, x_n$, maintaining the increasing order. The process is repeated recursively until the final sum is obtained. In particular cases the insertion

method reduces to one of the other two. For example, if $x_i = 2^{i-1}$, the insertion method is equivalent to recursive summation, since the insertion is always to the bottom of the list:

$$1248 \quad \rightarrow \quad \underline{3}48 \quad \rightarrow \quad \underline{7}8 \quad \rightarrow \quad 15.$$

On the other hand, if $1 \leq x_1 < x_2 < \ldots < x_n \leq 2$, every insertion is to the end of the list, and the method is equivalent to pairwise summation if n is a power of 2; for example, if $0 < \epsilon < 1/2$,

$$1,\ 1+\epsilon,\ 1+2\epsilon,\ 1+3\epsilon \quad \rightarrow \quad 1+2\epsilon,\ 1+3\epsilon,\ \underline{2+\epsilon} \quad \rightarrow \quad 2+\epsilon,\ \underline{2+5\epsilon} \quad \rightarrow \quad 4+6\epsilon.$$

To choose between these met hods we need error analysis, which we develop in the next section.

4.2. Error Analysis

Error analysis can be done individually for the recursive, pairwise and insertion summation methods, but it is more profitable to recognize that each is a special case of a general algorithm and to analyse that algorithm.

Algorithm 4.1. Given numbers $x_1, \ldots, x_n$ this algorithm computes $\boldsymbol{S}_n = \sum_{i=1}^{n} x_i$.

```
Let S = {x_1,...,x_n}.
repeat while S contains more than one element
        Remove two numbers x and y from S,
        and add their sum x + y to S.
end
Assign the remaining element of S to S_n .
```

Note that since there are n numbers to be added and hence n - 1 additions to be performed, there must be precisely n - 1 executions of the repeat loop.

First, let us check that the previous methods are special cases of Algorithm 4.1. Recursive summation (with any desired ordering) is obtained by taking x at each stage to be the sum computed on the previous stage of the algorithm. Pairwise summation is obtained by $[\log_2 n]$ groups of executions of the repeat loop, in each group of which the members of $\boldsymbol{S}$ are broken into pairs, each of which is summed. Finally, the insertion method is, by definition, a special case of Algorithm 4.1.

Now for the error analysis. Express the ith execution of the repeat loop as $T_i = xi_1 + yi_1$. The computed sums satisfy (using (2.5))

$$\widehat{T}_i = \frac{x_{i_1} + y_{i_1}}{1 + \delta_i}, \quad |\delta_i| \leq u, \quad i = 1{:}n-1. \tag{4.1}$$

The local error introduced in forming $\widehat{T}_i$ is $\delta_i \widehat{T}_i$. The overall error is the sum of the local errors (since summation is a linear process), so overall we have

$$E_n := S_n - \widehat{S}_n = \sum_{i=1}^{n-1} \delta_i \widehat{T}_i. \tag{4.2}$$

The smallest possible error bound is therefore

$$|E_n| \le u \sum_{i=1}^{n-1} |\widehat{T}_i|. \tag{4.3}$$

(This is actually in the form of a running error bound, because it contains the computed quantities-see §3.3.) It is easy to see that $|\widehat{T}_i| \le \sum_{j=1}^{n} |x_j| + O(u)$ for each i, and so we have also the weaker bound

$$|E_n| \le (n-1)u \sum_{i=1}^{n} |x_i| + O(u^2). \tag{4.4}$$

This is a forward error bound. A backward error result showing that $\widehat{S}_n$ is the exact sum of terms $x_i(1+\epsilon_i)$ with $|\epsilon_i| \le \gamma_{n-1}$ can be deduced from (4.1), using the fact that no number x_i takes part in more than n - 1 additions.

The following criterion is apparent from (4.2) and (-1.3):

> In designing or choosing a summation met hod to achieve high accuracy, the aim should be to minimize the absolute values of the intermediate sums T_i.

The aim specified in this criterion is surprisingly simple to state. When we consider specific methods, however, we find that the aim is difficult to achieve.

Consider recursive summation, for which $T_{i-1} = S_i := \sum_{j=1}^{i} x_j$. Ideally, we would like to choose the ordering of the x_i to minimize $\sum_{i=2}^{n} |\widehat{S}_i|$. This is a combinatorial optimization problem that is too expensive to solve in the context of summation. A reasonable compromise is to determine the ordering sequentially by minimizing, in turn, $|x_1|$, $|S_2|$, . . . , $|S_{n\text{-}1}|$. This ordering strategy, which we denote by Psum, can be implemented with $O(n \log n)$ comparisons. If we are willing to give up the property that the ordering is influenced by the signs of the x_i we can instead use the increasing ordering, which in general will lead to a larger value of $\sum_{i=2}^{n} |\widehat{S}_i|$ than that for the Psum ordering. If all the x_i have the same sign then all these orderings are equivalent. Therefore *when summing nonnegative numbers by recursive summation the increasing ordering is the best ordering, in the sense of having the smallest a priori forward error bound.*

How does the decreasing ordering fit into the picture? For the summation of positive numbers this ordering has little to recommend it. The bound (4.3) is no smaller, and potentially much larger, than it is for the increasing ordering. Furthermore, in a sum of positive terms that vary widely in magnitude the decreasing ordering may not allow the smaller terms to contribute to the sum (which is why the harmonic sum $\sum_{i=1}^{n} 1/i$ "converges" in floating point arithmetic as $n \to \infty$). However, consider the example with $n = 4$ and

$$x = [1,\ M,\ 2M,\ -3M], \tag{4.5}$$

where M is a floating point number so large that $fl(1 + M) = M$ (thus $M > u^{-1}$). The three orderings considered so far produce the following results:

$$\begin{aligned} \text{Increasing:} \quad & \widehat{S}_n = fl(1 + M + 2M - 3M) = 0, \\ \text{Psum:} \quad & \widehat{S}_n = fl(1 + M - 3M + 2M) = 0, \\ \text{Decreasing:} \quad & \widehat{S}_n = fl(-3M + 2M + M + 1) = 1. \end{aligned}$$

Thus the decreasing ordering sustains no rounding errors and produces the exact answer, while both the increasing and Psum orderings yield computed sums with relative error 1. The reason why the decreasing ordering performs so well in this example is that it adds the "1" after the inevitable heavy cancellation has taken place, rather than before, and so retains the important information in this term. If we evaluate the term $\mu = \sum_{i=2}^{n} |\widehat{S}_i|$ in the error bound (4.3) for example (4.5) we find

$$\text{Increasing: } \mu = 4M, \quad \text{Psum: } \mu = 3M, \quad \text{Decreasing: } \mu = M + 1,$$

so (4.3) "predicts" that the decreasing ordering will produce the most accurate answer, but the bound it provides is extremely pessimistic since there are no rounding errors in this instance.

Extrapolating from this example, we conclude that the decreasing ordering is likely to yield greater accuracy than the increasing or Psum orderings whenever there is heavy cancellation in the sum, that is, whenever $|\sum_{i=1}^{n} x_i| \ll \sum_{i=1}^{n} |x_i|$.

Turning to the insertion method, a good explanation of the insertion strategy is that it attempts to minimize, one at a time, the terms $|\widehat{T}_1|, \ldots, |\widehat{T}_{n-1}|$ in the error bound (4.3). Indeed, if the x_i are all nonnegative the insertion method minimizes this bound over all instances of Algorithm 4.1.

Finally, we note that a stronger form of the bound (4.4) holds for pairwise summation. It can be deduced from (4.3) or derived directly, as follows. Assume for simplicity that $n = 2^r$. Unlike in recursive summation each addend takes part in the same number of additions, $\log_2 n$. Therefore we have a relation of the form

$$\widehat{S}_n = \sum_{i=1}^{n} x_i \prod_{k=1}^{\log_2 n} (1 + \delta_k^{(i)}), \qquad |\delta_k^{(i)})| \le u,$$

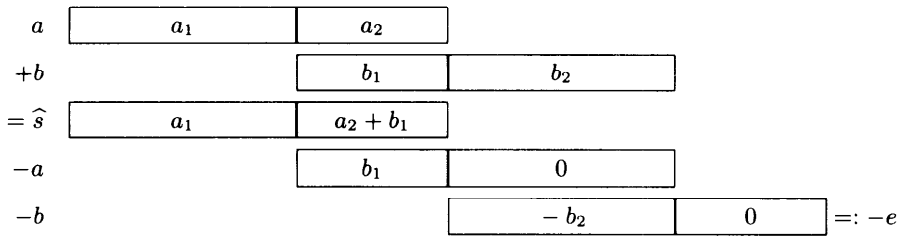

Figure 4.1. *Recovering the rounding error.*

which leads to the bound

$$|E_n| \le \gamma_{\log_2 n} \sum_{i=1}^{n} |x_i|. \tag{4.6}$$

Since it is proportional to $\log_2 n$ rat her than n, this is a smaller bound than (4.4), which is the best bound of this form that holds in general for Algorithm 4.1.

4.3. Compensated Summation

We have left to last the *compensated summation* method, which is recursive summation with a correction term cleverly designed to diminish the rounding errors. Compensated summation is worth considering whenever an accurate sum is required and computations are already taking place at the highest precision supported by the hardware or the programming language in use.

In 1951 Gill [449, 1951] noticed that the rounding error in the sum of two numbers could be estimated by subtracting one of the numbers from the sum, and he made use of this estimate in a Runge-Kutta code in a program library for the EDSAC computer. Gill's estimate is valid for fixed point arithmetic only. Kahan [625, 1965] and Møller [777, 1965] both extended the idea to floating point arithmetic. Møller shows how to estimate $a + b - fl(a + b)$ in chopped arithmetic, while Kahan uses a slightly simpler estimate to derive the compensated summation method for computing $\sum_{i=1}^{n} x_i$.

The estimate used by Kahan is perhaps best explained with the aid of a diagram. Let a and b be floating point numbers with $|a| \ge |b|$, let $\widehat{s} = fl(a+b)$, and consider Figure 4.1, which uses boxes to represent the mantissas of a and b. The figure suggests that if we evaluate

$$e = -\big[\big((a+b) - a\big) - b\big] = (a - \widehat{s}) + b$$

in floating point arithmetic, in the order indicated by the parentheses, then the computed ê will be a good estimate of the error $(a + b) - \hat{s}$. In fact, for rounded floating point arithmetic in base 2, we have

$$a + b = \hat{s} + \hat{e}, \tag{4.7}$$

that is, the computed $\hat{e}$ represents the error exactly. This result (which does not hold for all bases) is proved by Dekker [275, 1971, Thm. 4.7], Knuth [668, 1981, Thm. C, p. 221], and Linnainmaa [703, 1974, Thm. 3]. Note that there is no point in computing $fl(\hat{s} + \hat{e})$, since $\hat{s}$ is already the best floating point representation of $a + b$!

Kahan's compensated summation method employs the correction e on every step of a recursive summation. After each partial sum is formed, the correction is computed and immediately added to the next term x_i before that term is added to the partial sum. Thus the idea is to capture the rounding errors and feed them back into the summation. The method may be written as follows.

Algorithm 4.2 (compensated summation). Given floating point numbers $x_1, \ldots, x_n$ this algorithm forms the sum $= \sum_{i=1}^{n} x_i$ by compensated summation.

```
s = 0 ; e = 0
for i = 1:n
    temp = s
    y = x i + e
    s = t e m p + y
    e = (temp - s) + y % Evaluate in the order shown.
end
```

The compensated summation method has two weaknesses: $\hat{e}$ is not necessarily the exact correction, since (4.7) is based on the assumption that $|a| \geq |b|$, and the addition $y = x_i + e$ is not performed exactly. Nevertheless, the use of the corrections brings a benefit in the form of an improved error bound. Knuth [668, 1981, Ex. 19, pp. 229, 572-573] shows that the computed sum $\hat{S}_n$ satisfies

$$\hat{S}_n = \sum_{i=1}^{n} (1 + \mu_i) x_i, \qquad |\mu_i| \leq 2u + O(nu^2), \tag{4.8}$$

which is an almost ideal backward error result (a more detailed version of Knuth's proof is given by Goldberg [457, 1991]).

In [627, 1972] and [628, 1973] Kahan describes a variation of compensated summation in which the final sum is also corrected (thus "$s = s + e$" is appended to the algorithm above). Kahan states in [627, 1972] and proves in

[628, 1973] that (4.8) holds with the stronger bound $|\mu_i| \leq 2u+O((n-i+1)u^2)$ The proofs of (4.8) given by Knuth and Kahan are similar; they use the model (2.4) with a subtle induct ion and some intricate algebraic manipulation.

The forward error bound corresponding to (4.8) is

$$|E_n| \leq \left(2u + O(nu^2)\right) \sum_{i=1}^{n} |x_i|. \tag{4.9}$$

As long as $nu \leq 1$, the constant in this bound is independent of n, and so the bound is a significant improvement over the bounds (4.4) for recursive summation and (4.6) for pairwise summation. Note, however, that if $\sum_{i=1}^{n} |x_i| \gg |\sum_{i=1}^{n} x_i|$, compensated summation is not guaranteed to yield a small relative error.

Another version of compensated summation has been investigated by several authors: Jankowski, Smoktunowicz, and Wozniakowski [609, 1983], Jankowski and Wozniakowski [611, 1985], Kielbasinski [655, 1973], Neumaier [788, 1974], and Nickel [797, 1970]. Here, instead of immediately feeding each correction back into the summation, the correct ions are accumulated separately by recursive summation and then the global correction is added to the computed sum. For this version of compensated summation Kielbasinski [655, 1973] and Neumaier [788, 1974] show that

$$\widehat{S}_n = \sum (1 + \mu_i) x_i, \qquad |\mu_i| \leq 2u + n^2 u^2, \tag{4.10}$$

provided $nu \leq 0.1$; this is weaker than (4.8) in that the second-order term has an extra factor n. If $n^2 u \leq 0.1$ then in (4.10), $|\mu_i| \leq 2.1u$. Jankowski, Smoktunowicz, and Wozniakowski [609, 1983] show that, by using a divide and conquer implementation of compensated summation, the range of n for which $|\mu_i| \leq cu$ holds in (4.10) can be extended, at the cost of a slight increase in the size of the constant c.

Neither the correction formula (4.7) nor the result (4.8) for compensated summation holds under the no-guard-digit model of floating point arithmetic. Indeed, Kahan [634, 1990] constructs an example where compensated summation fails to achieve (4.9) on certain Cray machines, but he states that such failure is extremely rare. In [627, 1972] and [628, 1973] Kahan gives a modification of the compensated summation algorithm in which the assignment "$e = (temp - s) + y$" is replaced by

```
f = 0
if sign(temp) = sign(y), f = (0.46 * s - s) + s, end
e = ((temp - f) - (s - f)) + y
```

Kahan shows in [628, 1973] that the modified algorithm achieves (4.8) "on all North American machines with floating hardware" and explains that "The mysterious constant 0.46, which could perhaps be any number between 0.25 and 0.50, and the fact that the proof requires a considerat ion of known machines designs, indicate that this algorithm is not an advance in computer science."

Viten'ko [1056, 1968] shows that under the no-guard-digit model (2.6) the summation met hod with the optimal error bound (in a certain sense defined in [1056, 1968]) is pairwise summation. This does not contradict Kahan's result because Kahan uses properties of the floating point arithmetic beyond those in the no-guard-digit model.

A good illustration of the benefits of compensated summation is provided by Euler's method for the ordinary differential equation initial value problem $y' = f(x,y)$, $y(a)$ given, which generates an approximate solution according to $y_{k+1} = y_k + hf_k$, $y_0 = y(a)$. We solved the equation $y' = -y$ with $y(0) = 1$ over [0,1] using n steps of Euler's method ($nh = 1$), with n ranging from 10 to 10^8. With compensated summation we replace the statements $x = x + h$, $y = y + h * f(x,y)$ by (with the initialization $cx = 0$, $cy = 0$)

```
dx = h + cx
new-x = x + dx
cx = (x - new-x) + dx
x = new_x

dy = h * f(x,y) + cy
new-y = y + dy
cy = (y - new-y) + dy
y = new-y
```

Figure 4.2 shows the errors $e_n = |y(1) - \hat{y}_n|$ where $\hat{y}_n$ is the computed approximation to $y(1)$. The computations were done in Fortran 90 in single precision arithmetic on a Sun SPARCstation ($u \approx 6 \times 10^{-8}$). Since Euler's method has global error of order h, the error curve on the plot should be approximately a straight line. For the standard. implement at ion of Euler's method the errors e_n start to increase steadily beyond $n = 20{,}000$ because of the influence of rounding errors. With compensated summation the errors e_n are much less affected by rounding errors and do not grow in the range of n shown (for $n = 10^8$, e_n is about 10 times larger than it would be in exact arithmetic). Plots of U-shaped curves showing total error against stepsize are common in numerical analysis textbooks (see, e.g., Forsythe, Malcolm, and Moler [395, 1977, p. 119] and Shampine [910, 1994, p. 259]), but the textbooks rarely point out that the "U" can be flattened out by compensated summation.

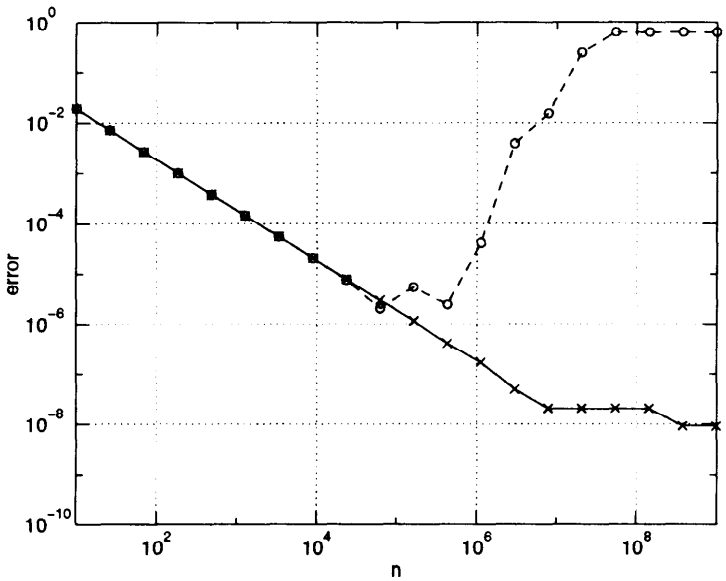

Figure *4.2. Errors* $|y(1) - \widehat{y}_n|$ *for Euler's method with* ("×") *and without* ("o") *compensated summation.*

The cost of applying compensated summation in an ordinary differential equation solver is almost negligible if the function f is at all expensive to evaluate. But, of course, the benefits it brings are noticeable only when a vast number of integration steps are taken. Very long-term integrations are undertaken in celestial mechanics, where roundoff can affect the ability to track planetary orbits. Researchers in astronomy use compensated summation, and other techniques, to combat roundoff. An example application is a 3 million year integration of the planets in the solar system by Quinn, Tremaine, and Duncan [854, 1991]; it used a linear multistep method of order 13 with a constant stepsize of 0.75 days and took 65 days of machine time on a Silicon Graphics 4D-25 workstation. See also Quinn and Tremaine [853, 1991] and Quinlan [851, 1994].

Finally, we describe an even more ingenious algorithm called *doubly compensated summation,* derived by Priest [844, 1992] from a related algorithm of Kahan. It is compensated summation with 2 extra applications of the correction process[11] and it requires 10 instead of 4 additions per step. The algorithm is tantamount to simulating double precision arithmetic with single precision arithmetic; it requires that the summands first be sorted into

[11]The algorithm should perhaps be called triply compensated summation, but we adopt Priest's terminology.

decreasing order, which removes the need for certain logical tests that would otherwise be necessary.

Algorithm 4.3 (doubly compensated summation). Given floating point numbers $x_1, \ldots, x_n$, this algorithm forms the sum $s_n = \sum_{i=1}^{n} x_i$ by doubly compensated summation. All expressions should be evaluated in the order specified by the parentheses.

```
Sort the x_i so that |x_i| ≥ |x_2| ≥ . . . ≥ |x_n|.
s_1 = x_1;  c_1 = 0
for i = 2:n
    y_k = c_{k-1} + x_k
    u_k = x_k - (y_k - c_{k-1})
    t_k = y_k + s_{k-1}
    v_k = y_k - (t_k - s_{k-1})
    z_k = u_k + v_k
    s_k = t_k + z_k
    c_k = z_k - (s_k - t_k)
end
```

Priest [844, 1992, §4.1] analyses this algorithm for t-digit base β arithmetic that satisfies certain reasonable assumptions-ones which are all satisfied by IEEE arithmetic. He shows that if $n \le \beta^{t-3}$ then the computed sum $\widehat{s}_n$ satisfies

$$|s_n - \widehat{s}_n| \le 2u|s_n|,$$

that is, the computed sum is accurate virtually to full precision.

4.4. Other Summation Methods

We mention briefly two further classes of summation algorithms. The first builds the sum in a series of accumulators, which are themselves added to give the sum. As originally described by Wolfe [1107, 1964] each accumulator holds a partial sum lying in a different interval. Each term x_i is added to the lowest-level accumulator; if that accumulator overflows it is added to the next-highest one and then reset to zero, and this cascade continues until no overflow occurs. Modifications of Wolfe's algorithm are presented by Malcolm [723, 1971] and Ross [880, 1965]. Malcolm [723, 1971] gives a detailed error analysis to show that his method achieves a relative error of order u. A drawback of the algorithm is that it is strongly machine dependent. An interesting and crucial feature of Malcolm's algorithm is that on the final step the accumulators are summed by recursive summation in order of *decreasing* absolute value, which in this particular situation precludes severe loss of significant digits and guarantees a small relative error.

Another class of algorithms, referred to as "distillation algorithms" by Kahan [633, 1987], work as follows: given $x_i = fl(x_i)$, $i = 1:n$, they iteratively construct floating point numbers $x_1^{(k)}, \ldots, x_n^{(k)}$ such that $\sum_{i=1}^n x_i^{(k)} = \sum_{i=1}^n x_i$, terminating when $x_n^{(k)}$approximates $\sum_{i=1}^n x_i$ with relative error at most u. Kahan states that these algorithms appear to have average run times of order at least n log n. See Bohlender [130, 1977], Kahan [633, 1987], Leuprecht and Oberaigner [700, 1982], Pichat [830, 1972], and Priest [844, 1992, pp. 66-69) for further details and references.

4.5. Statistical Estimates of Accuracy

The rounding error bounds presented above can be very pessimistic, because they account for the worst-case propagation of errors. An alternative way to compare summation methods is through statistical estimates of the error, which may be more representative of the average case. A statistical analysis of three summation methods has been given by Robertazzi and Schwartz [874, 1988] for the case of nonnegative x_i. They assume that the relative errors in floating point addition are statistically independent and have zero mean and finite variance σ^2. Two distributions of nonnegative x_i are considered: the uniform distribution on $[0,2\mu]$. and the exponential distribution with mean μ. Making various simplifying assumptions Robertazzi and Schwartz estimate the mean square error (that is, the variance of the absolute error) of the computed sums from recursive summation with random, increasing and decreasing orderings, and from insertion summation and pair-wise summation (with the increasing ordering). Their results for the summation of n numbers are given in Table -1.1.

The results show that for recursive summation the ordering affects only the constant in the mean square error, with the increasing ordering having the smallest constant and the decreasing ordering the largest; since the x_i are nonnegative, this is precisely the ranking given by the rounding error bound (4.3). The insertion and pairwise summation methods have mean square errors proportional to n^2 rather than n^3 for recursive summation, and the insertion method has a smaller constant than pairwise summation. This is also consistent with the rounding error analysis, in which for nonnegative x_i the insertion met hod satisfies an error bound no larger than pairwise summation and the latter method has an error bound with a smaller constant than for recursive summation ($\log_2 n$ versus n).

4.6. Choice of Method

There is a wide variety of summation methods to choose from. For each met hod the error can vary greatly with the data, within the freedom afforded

Table 4.1. *Mean square errors for nonnegative x_i.*

Distrib.	Increasing	Random	Decreasing	Insertion	Pairwise
Unif(0,2μ)	$0.20\mu^2 n^3 \sigma^2$	$0.33\mu^2 n^3 \sigma^2$	$0.53\mu^2 n^3 \sigma^2$	$2.6\mu^2 n^2 \sigma^2$	$2.7\mu^2 n^2 \sigma^2$
EXP(μ)	$0.13\mu^2 n^3 \sigma^2$	$0.33\mu^2 n^3 \sigma^2$	$0.63\mu^2 n^3 \sigma^2$	$2.6\mu^2 n^2 \sigma^2$	$4.0\mu^2 n^2 \sigma^2$

by the error bounds; numerical experiments show that, given any two of the methods, data can be found for which either method is more accurate than the other [553, 1993]. However, some specific advice on the choice of met hod can be given.

1. If high accuracy is important, consider implementing recursive summation in higher precision; if feasible this may be less expensive (and more accurate) than using one of the alternative methods at the working precision. What can be said about the accuracy of the sum computed at higher precision? If $S_n = \sum_{i=1}^n x_i$ is computed by recursive summation at double precision (unit roundoff u^2) and then rounded to single precision, an error bound of the form $|S_n - \widehat{S}_n| \le u|\widehat{S}_n| + nu^2 \sum_{i=1}^n |x_i|$ holds. Hence a relative error of order u is guaranteed if $nu \sum_{i=1}^n |x_i| \le |S_n|$. Priest [844, 1992, pp. 62-63] shows that if the x_i are sorted in decreasing order of magnitude before being summed in double precision, then $|S_n - \widehat{S}_n| \le 2u|S_n|$ holds provided only that $n \le \beta^{t-3}$ for t-digit base β arithmetic satisfying certain reasonable assumptions. Therefore the decreasing ordering may be worth arranging if there is a lot of cancellation in the sum. An alternative to extra precision computation is doubly compensated summation, which is the only other method described here that guarantees a small relative error in the computed sum.

2. For most of the methods the errors are, in the worst case, proportional to n. If n is very large, pairwise summation (error constant $\log_2 n$) and compensated summation (error constant of order 1) are attractive.

3. If the x_i all have the same sign then all the methods yield a relative error of at most nu and compensated summation guarantees perfect relative accuracy (as long as $nu \le 1$). For recursive summation of one-signed data, the increasing ordering has the smallest error bound (4.3) and the insertion method minimizes this error bound over all instances of Algorithm 4.1.

4. For sums with heavy cancellation $(\sum_{i=1}^n |x_i| \gg |\sum_{i=1}^n x_i|)$, recursive summation with the decreasing ordering is attractive, although it cannot be guaranteed to achieve the best accuracy.